Transport One - Full Bug and Risk Audit

Audit date: 25 October 2026

Scope: Flutter app, Android configuration, Firebase Auth/Firestore, Cloudflare OTP/Protrack proxy, maps, trip workflow, reports/export, product catalogue and automated tests.

Constraint followed: application source was not changed.

Executive result

- Dart static analysis: PASS, no analyzer findings.
- Product catalogue: PASS for 256 records; no missing referenced images, duplicate IDs or empty required fields.
- Deployed OTP Worker health: PASS (`HTTP 200`).
- GPS proxy authorization boundary: PASS for anonymous rejection (`HTTP 401`).
- Full Flutter test suite: FAILS TO COMPLETE. It reached 91 tests, emitted plugin/binding warnings, and then hung until manually stopped.
- Production readiness: NOT READY. Core backend provisioning and cross-device persistence are incomplete.

Critical findings

C1. Live GPS cannot work in the current APK

Evidence:

- `ProtrackGpsService.proxyUrl` defaults to an empty string in `lib/src/services/protrack_gps_service.dart`.
- `TRANSPORT_ONE_FIXED_BUILD31.apk` contains the OTP Worker URL but no `PROTRACK_PROXY_URL` or Cloud Functions URL.
- When the proxy URL is empty, the service explicitly returns `Secure Protrack proxy is not configured`.

Impact: IMEI values alone cannot produce live location. The APK cannot call the deployed Protrack endpoint.

Required fix: deploy/configure a trusted Protrack proxy, supply `PROTRACK_PROXY_URL` at release build time, ensure Protrack Open API is enabled by the vendor, then test each assigned IMEI with a real Firebase driver token.

C2. Add Driver can report success without creating a real account

Evidence:

- `FleetAdminService.createDriver()` returns immediately when `FLEET_ADMIN_API_URL` is empty.
- The controller then adds the driver locally and saves it even though no Firebase Auth user was created.
- The current APK does not contain a fleet-admin backend URL.

Impact: Admin sees a driver in the app, but that driver cannot sign in on another device.

Required fix: make missing backend a blocking error, configure/deploy `adminApi`, and only commit the local profile after the backend confirms Firebase Auth user + claims + Firestore profile creation.

C3. Offline/cloud write failures are shown as successful operations

Evidence:

- `FirebaseBackendService._put()` returns `true` when Firebase is not configured.
- Controller calls to `syncTripRecord`, `syncExpenseRecord`, `syncBillingRecord`, `syncAttendanceRecord` and `syncDeliveryProof` generally ignore their returned boolean.

Impact: records appear saved locally while cloud persistence silently failed. Users can lose data when changing/reinstalling devices.

Required fix: return an explicit state (`synced`, `queuedOffline`, `failed`), display it in UI, persist a retry queue, and surface final sync errors.

C4. Cloud synchronization is one-way and not restored into the app

Evidence:

- A generic `fetchCollection()` exists but has no production call sites.
- Startup loads trips, drivers, attendance, expenses, billing and proofs from local preferences, not Firestore.

Impact: data does not reliably follow an account to another phone; server-side updates are not reflected; local data can diverge from Firestore.

Required fix: implement role-filtered startup hydration, conflict resolution, realtime listeners or controlled refresh, pagination and last-sync metadata.

C5. WhatsApp OTP session is not a Firebase session

Evidence:

- Cloudflare `/verify` returns a custom HMAC session token and an `admin` user object.
- `loginWithTrustedWhatsAppOtp()` accepts that response for the current runtime.
- Startup restoration deliberately restores only Firebase Auth and clears the WhatsApp session.
- Firestore and Protrack services request a Firebase ID token, so the custom OTP session cannot access them.

Impact: WhatsApp login may open the UI temporarily, but Firebase/Protrack operations fail and login disappears after restart.

Required fix: after verified OTP, mint a Firebase custom token through a trusted backend and sign in with `FirebaseAuth.signInWithCustomToken`; use Firebase claims as the single role authority.

C6. Real-time traffic presentation contains fabricated segment speeds/locations

Evidence:

- `calculateRoadTrafficSegments()` hardcodes corridor speeds such as 78, 42, 22 and 18 km/h.
- `calculateRoadHotspots()` hardcodes 76/24/45 km/h and generic locations such as `Toll Plaza / Flyover Approach`.

Impact: UI can look like live road telemetry when it is only an estimate, creating incorrect operational decisions.

Required fix: show estimates clearly, or replace them with licensed traffic-provider segment data. Never label synthetic values as live.

High-severity findings

H1. Streetlight/night map mode is internally inconsistent

- `MapViewStyle.streetlight` and Carto dark tiles still exist.
- The main Google map converts every non-satellite style to `MapType.normal`; therefore streetlight cannot render there.
- Trip creation exposes only Street/Satellite, while other compact controls can cycle through streetlight.

Fix: either remove streetlight everywhere or implement one consistent supported renderer and UI state.

H2. Duplicate map stacks increase regressions

The app contains Google Maps in `map_workspace.dart` and `trips_page.dart`, plus a separate Flutter Map implementation in `trip_map_dialog.dart`. Pinning, zoom limits, route styling, failure handling and map type behavior differ.

Fix: create one reusable map component/controller contract for all screens.

H3. Map tile failure UI is not connected to Google Maps tile errors

`_mapTilesFailed` UI exists, but Google Maps does not expose the tile callback used by the older tile-map path. Users may get a blank/API-key screen without the intended retry state.

Fix: detect `onMapCreated` timeout, platform-channel errors and API authorization errors; show a diagnostic state with key/package/SHA checks.

H4. Trip creation uses long-press only

Trip map pins pickup/drop/stops only in `onLongPress`. This is easy to miss and accessibility-unfriendly; a normal tap does nothing.

Fix: retain long-press if desired but add explicit Pickup/Drop modes, visible confirmation and address preview.

H5. Firebase configuration failure is swallowed

`FirebaseBootstrapService.initialize()` stores the error and only prints it. The app can continue into login with generic failure messages.

Fix: expose a blocking setup screen containing the exact project/provider/configuration problem and a retry action.

H6. Weak driver-password scheme

Passwords based on `driver-name + last phone digits` are predictable and derived from known personal information.

Fix: generate random temporary passwords or use OTP/password-reset enrollment, force first-login change and apply Firebase password policy.

H7. Google Maps key exposure/restriction risk

The Maps key is correctly loaded from an ignored local properties file, but Android keys are present in APKs by design and must be restricted server-side.

Fix: restrict the key to package `com.transportone.transport_one`, correct release SHA-1/SHA-256 and only required Maps APIs; set budgets/alerts and rotate any broadly unrestricted key.

H8. Release build depends on undocumented manual dart-defines

Firebase Web, GPS proxy, routing proxy, fleet admin and AI endpoints depend on build-time values. A missed define silently disables features.

Fix: use validated build flavors/config files and fail release builds when required production values are absent.

Medium findings

M1. Test suite hangs and hides lifecycle defects

The suite emitted `ServicesBinding has not yet been initialized`, `MissingPluginException` for secure storage/image picker and notification initialization warnings, then stopped progressing after test 91.

Fix: initialize test bindings, provide platform mocks, isolate singleton state/timers, add per-test timeouts and tear down GPS polling/streams.

M2. Extremely large source files

Examples: `map_workspace.dart` ~5500 lines, `planner_controller.dart` ~3764, `driver_portal_page.dart` ~3547, `trips_page.dart` ~3490 and `dashboard_page.dart` ~3412.

Impact: duplicated logic, hard reviews, fragile UI changes and slow testing.

Fix: split by feature/state/service; keep widgets small; replace the monolithic controller with repositories and feature controllers.

M3. Broad silent exception handling

Many `catch (_) {}` branches discard map, image, parsing, storage and export failures.

Fix: use typed exceptions, structured logging and actionable user messages; only suppress explicitly harmless failures.

M4. Custom local cryptography is difficult to validate

Operational data uses a custom HMAC-derived stream cipher labelled AEAD instead of a reviewed platform/library primitive.

Fix: use Android Keystore-backed AES-GCM/ChaCha20-Poly1305 from a maintained library; version and migrate stored data.

M5. Local record filtering may delete legitimate data

On load, drivers not matching the build-time bootstrap phone/name list are discarded. A legitimately created backend driver can disappear locally after restart if bootstrap data is stale.

Fix: treat backend profiles as source of truth and bootstrap only once for initial migration.

M6. Firestore write schema validation is incomplete

Rules validate telemetry coordinates but most collections lack strict allowed-field, type, length and immutable-field validation.

Fix: add `keys().hasOnly(...)`, field types/ranges, timestamp checks and role-specific immutable fields; test rules with the emulator.

M7. No robust offline sync queue

Local save happens immediately, but failed cloud writes are not queued with retry/backoff/idempotency metadata.

Fix: add outbox records, exponential backoff, connectivity-triggered retry and a visible unsynced count.

M8. Export works only for currently available local/filter data

CSV, Excel and PDF generators exist and correctly block empty exports, but because cloud hydration is missing, exports may omit records saved on other devices.

Fix: fetch authoritative paginated data first, show data range/count and include export metadata/time zone.

M9. OTP service naming and API are misleading

`hasApiKey` only checks whether the URL starts with HTTPS, while `configure(key:)` ignores the key. This can mislead diagnostics.

Fix: rename to `isEndpointConfigured`, remove unused key input and add an authenticated `/health/config` diagnostic that reveals no secrets.

M10. Cloudflare CORS reflects arbitrary origins

Worker responses mirror the request Origin. Auth checks reduce impact, but browser access is broader than necessary.

Fix: allow only production origins and local development origins; return no CORS header for unknown origins.

Low/UI findings

- Long text and bilingual labels need tests at large font scale and narrow phones.
- Fixed overlays and bottom navigation can cover map controls/content on short screens.
- Several user-facing errors are generic and omit the failed subsystem/error code.
- No visible sync status differentiates local-only, queued and cloud-synced records.
- Report/export filenames should include timezone-safe timestamps and active filters.
- No automated golden tests cover the exact map/catalog overflow screenshots previously observed.
- Product catalogue is valid, but 256 bundled images increase APK size and memory pressure; thumbnails should be resized/cached and full images decoded only on demand.

Recommended execution order

1. Configure and verify Firebase Authentication plus admin custom claims.
2. Deploy/configure `adminApi`; block fake local success.
3. Deploy/configure Protrack proxy; confirm vendor Open API entitlement.
4. Replace WhatsApp custom UI session with Firebase custom-token login.
5. Implement two-way Firestore hydration and offline outbox.
6. Remove fabricated live traffic values or clearly mark estimates.
7. Unify map implementations and fix pin/gesture/error behavior.
8. Fix hanging tests and add integration/golden/Firestore-rules tests.
9. Harden release config, API-key restrictions and monitoring.
10. Refactor oversized files after behavior is covered by tests.

Copy-paste implementation prompt

Audit and repair this Flutter fleet app without changing its visual design unnecessarily. Work from evidence, not assumptions. First run dart analyze and the complete Flutter test suite with timeouts. Preserve user data and existing features.

Priority requirements:

1) Production auth: Firebase Auth is the only session/role authority. Convert verified WhatsApp OTP into a Firebase custom-token sign-in. Restore it across restart. Never trust client role fields.

2) Driver creation: if FLEET_ADMIN_API_URL/adminApi is missing or fails, show a blocking error and do not add the driver locally. Backend must create/update Firebase Auth, custom claims and Firestore profile atomically/idempotently.

3) GPS: require PROTRACK_PROXY_URL in release; never call vendor API from client. Verify role/assigned IMEIs server-side. Show exact states: not configured, Open API disabled, unauthorized, no device, stale fix, online. No mock telemetry.

4) Persistence: implement role-filtered Firestore startup hydration, pagination, conflict handling and an encrypted offline outbox with retry/backoff. Distinguish local-only/queued/synced/failed. Never return success merely because Firebase is absent.

5) Maps: unify Live Route, Trip Creator and Trip Details behind one reusable Google Maps component. Preserve satellite labels and live traffic. Make pickup/drop/via selection reliable with explicit modes plus long-press, reverse-geocode names, draggable/editable markers and blue road-following route. Handle API-key/package/SHA/network errors visibly. Remove or fully implement inconsistent streetlight mode.

6) Traffic: delete hardcoded fake speeds/hotspots. Use licensed provider data or label estimates prominently as estimates.

7) Reports: export authoritative filtered data to PDF/XLSX/CSV, block empty exports, include filters/date range/timezone, and verify files open correctly.

8) Security: restrict Maps key by Android package + release SHA and APIs; enforce strict Firestore schemas/types/ownership; allowlist Worker CORS; use reviewed AEAD storage; never log secrets/tokens/IMEIs unmasked.

9) Tests: fix suite hang and plugin/binding warnings. Mock secure storage, notifications, image picker, Firebase and maps. Add Firestore emulator rules tests, auth/custom-claim tests, GPS proxy contract tests, offline-sync tests, map gesture tests, export validation and golden tests for narrow screens/large fonts.

10) Architecture: split files over ~1000 lines into feature widgets/controllers/repositories only after tests protect behavior.

Acceptance criteria:

- dart analyze: zero issues.
- all tests complete and pass; no hangs or uncaught plugin/binding warnings.
- release build fails fast when required production config is absent.
- newly created driver can sign in on another device and sees only assigned vehicle.
- real GPS position updates through authenticated proxy and stale data is labelled.
- trip/expense/attendance/proof survives reinstall/new device via Firestore.
- map supports pan/zoom/tap/long-press and renders route/satellite/traffic correctly.
- PDF/XLSX/CSV exports contain the same filtered record count.
- provide a final change log, test evidence, remaining external blockers and APK path.

External blockers that code alone cannot solve

- Protrack dealer must enable/permit Open API access for the account.
- Firebase Authentication providers and custom claims must be configured.
- Cloud Functions generally require billing; Cloudflare can host compatible endpoints if configured securely.
- Google Maps key restrictions and APIs must be correct in Google Cloud Console.
- Real GPS accuracy depends on powered tracker, SIM data, sky visibility and fresh vendor telemetry.